

CSCE 620 COMPUTATIONAL GEOMETRY · MODULE 02 · LECTURE NOTES

Exact and Adaptive Predicates

Exactness is affordable. Here is the bill.

MODULE OUTCOMES

Explain why the predicates are integer problems in disguise.

Use a floating-point filter with a correct error bound.

Explain expansion arithmetic and adaptive evaluation.

Implement the incircle predicate.

Handle degeneracy by symbolic perturbation.

1 The filter

THE PREDICATE IS A POLYNOMIAL, SO ITS SIGN IS DECIDABLE

Orientation is a degree-2 polynomial in the input coordinates with integer coefficients; the incircle predicate is degree 4. Neither involves division, square roots, or transcendental functions.

And a floating-point number is an exact rational. An IEEE double is precisely an integer times a power of two — there is nothing approximate about the *input*.

So the true value of the predicate is an exactly representable rational number, and determining its sign is a finite, decidable problem. It requires no approximation of any kind.

The approximation entered only when we chose to evaluate an exact expression in a fixed-width format. That is worth stating plainly, because it reframes the whole difficulty: this was never an ill-conditioned problem in the numerical-analysis sense. It is an exactly answerable question that we chose to answer inexactly, and §2 shows we can afford not to.

```
double orient_filtered(P a, P b, P c) {
    double detleft  = (a.x - c.x) * (b.y - c.y);
    double detright = (a.y - c.y) * (b.x - c.x);
    double det      = detleft - detright;
    double detsum;

    // Cancellation can only hurt when the two products share a sign.
    if      (detleft > 0) { if (detrigh <= 0) return det;
                        detsum = detleft + detright; }
    else if (detleft < 0) { if (detrigh >= 0) return det;
                        detsum = -detleft - detright; }
    else return det;

    // PROVEN forward error bound, not a tuned constant.
    double errbound = CCWERRBOUND_A * detsum;
    if (det >= errbound || -det >= errbound) return det;
    return orient_exact(a, b, c);          // taken on <1% of real input
}
```

A FILTER IS NOT AN EPSILON

An epsilon says: 'the computed value is close to zero, so I will call it zero.' That is a guess. It is wrong precisely when the true value is small and nonzero — which is exactly the case the epsilon was introduced to handle.

A filter says: 'the computed value exceeds the proven bound on the error, therefore its sign is certainly correct.' That is a deduction from the floating-point model, and it either applies or it does not.

And when the filter cannot certify the sign, it does not guess. It escalates to exact evaluation, which always terminates with the right answer.

So the answer is always exact. The filter changes only the cost. Module 01's entire table of non-solutions is avoided by this single distinction — and the reason the distinction is not more widely understood is that the two look identical in the source code, differing only in whether the threshold was derived or chosen.

2 The exact path

EXPANSION ARITHMETIC: EXACT VALUES AS SUMS OF DOUBLES

Represent an exact value as a sum of several doubles, ordered by magnitude and non-overlapping in their significands. The sum of the components is the exact value; each component is an ordinary machine number.

The primitives are two_sum and two_product, which compute the exact result of a single addition or multiplication as a high part and a low part. Each takes a handful of floating-point operations and contains no branches and no memory allocation.

So exact arithmetic here is ordinary floating-point arithmetic, carefully sequenced. There is no bignum library, no conversion to integers, no heap. The expansions live in a fixed-size stack array whose length is bounded by the degree of the polynomial.

Which is why it is fast enough to ship. Exact geometric predicates existed long before Shewchuk's 1997 paper; what the paper contributed was making them cheap enough that there is no longer a reason to use anything else, which is why it changed practice rather than only the literature.

- 1 **Compute the cheapest approximation and its error bound.** If the bound certifies the sign, return immediately. On non-degenerate input this happens essentially always.
- 2 **Otherwise refine:** compute the next correction term and add it, which both improves the estimate and tightens the error bound.
- 3 **Check again, and repeat.** Each successive stage costs more than the last and is required less often — the stages are needed in inverse proportion to their cost.
- 4 **The final stage is fully exact** and always determines the sign. On real input it is reached almost never; on deliberately degenerate input it is reached often, which is correct billing.

The measured result is roughly twice the cost of the naive predicate on average, and always correct. That figure is what settles the practical argument: a 2× cost on seven floating-point operations, in an algorithm that also does memory traffic and bookkeeping, is not a reason to accept wrong answers.

$$\text{incircle}(a,b,c,d) = \text{sign of the } 4 \times 4 \text{ determinant with rows } (p_x, p_y, p_x^2 + p_y^2, 1)$$

for each of the four points, with a, b, c in counter-clockwise order. It is positive when d lies inside the circle through the other three, negative outside, and **zero when all four are cocircular** — the degeneracy that makes the Delaunay triangulation non-unique (Module 09). **It is degree 4, so it cancels considerably worse than orientation** and its filter bound is correspondingly looser, meaning the exact path is taken more often. **Orientation and incircle are the only two predicates this course needs;** nearly everything else reduces to them.

3 Degeneracy, decided once

SYMBOLIC PERTURBATION REMOVES DEGENERACY BY DEFINITION

Imagine perturbing each input point by a distinct infinitesimal amount, with the perturbations ordered so that no two are comparable in magnitude — formally, each point i is moved by a different power of an infinitesimal ϵ .

Under such a perturbation no predicate can return zero. Any tie is broken by the perturbation, and the perturbed input is in general position by construction. **Degeneracy has been defined out of existence.**

And nothing is ever actually perturbed. When the exact predicate returns zero, a deterministic tie-breaking rule derived from the point indices supplies the sign that the infinitesimally perturbed input would have produced. The rule is a small amount of index comparison, evaluated only on the degenerate path.

So every degenerate case in every algorithm is handled by one rule rather than by special-case code in each. This is a large practical saving. **The output is a valid structure for an input infinitesimally close to yours** — which is usually exactly what is wanted, and occasionally is not, per the table below.

Approach	Cost	When it is right
Handle every degenerate case explicitly.	Substantially more code, and each special case is a fresh bug risk . The combinatorial explosion is real — segment intersection alone has a dozen degenerate configurations.	When the degenerate answer is itself the point — a CAD tool that must report coincident faces, or a validator.
Symbolic perturbation.	One rule, written once, applied everywhere. Slight cost on the degenerate path only.	The default, and the right choice for almost all of this course. You get a valid structure for a nearby input.
Reject degenerate input.	Trivial to implement.	Never, for real data. Module 01 §4: real authored geometry is saturated with degeneracy, so this rejects most of your input.

The caveat on perturbation is worth stating. It answers a subtly different question from the one asked: not *what is the structure of this input?* but *what is the structure of an input infinitesimally near this one?* If a user asks whether four points are cocircular, perturbing the answer away is simply wrong. Know which question your caller is asking.

USE SHEWCHUK'S CODE AFTER THIS MODULE

Write the predicates once yourself, to understand them. The filter's error-bound derivation in particular rewards doing by hand — it is the step that makes the difference between a filter and an epsilon concrete rather than verbal.

Then use predicates.c. It is public domain, it is embedded in CGAL, Triangle, TetGen, and a great many shipped systems, its error bounds are derived correctly, and it has been exercised against more adversarial input than you will generate.

Compiler flags can silently invalidate it. -ffast-math permits reassociation that breaks the error analysis; x87 excess precision on 32-bit x86 computes intermediates in 80 bits and rounds unpredictably; aggressive optimisation can contract a multiply-add into an FMA with different rounding. **All of these produce a predicate that is wrong in exactly the cases it exists to handle.**

So pin the flags, and test the predicate in the configuration you ship — not only in a debug build. This is a real failure mode that people hit, and the symptom is that the geometry works in debug and breaks in release, which sends everyone looking in the wrong place.

Where to see this taught differently

Original material only goes so far — hearing the same idea in another voice is how it sticks. These are free, and they cover this module specifically.

Resource	What it covers for this module
Shewchuk — Adaptive Precision Floating-Point Arithmetic and Fast Robust Geometric Predicates (free, with code)	The source for this entire module , including predicates.c. The error-bound derivations are in the paper and are worth following once.
Edelsbrunner & Mücke — Simulation of Simplicity (free)	The §3 technique, stated and proved by its authors.
Dekker; Knuth — two-sum and two-product	The expansion primitives of §2, with the correctness arguments.
CGAL — kernel and number type documentation (free)	How a production library packages exact predicates and inexact constructions, which is the Module 13 distinction.

Exercises

Do these in code, not on paper, unless the exercise says otherwise. The module is not finished until they run.

- 1 Implement two_sum and two_product and verify exactness against Fraction over many random inputs.
- 2 **Derive the error bound** for the naive orientation expression from the IEEE model. Compare your constant against Shewchuk's.

- 3 Implement the filtered orientation predicate with your bound.
- 4 **Measure how often the slow path is taken** on uniform random input, on lattice input, and on your degenerate generator from Module 01. Explain the three numbers.
- 5 Benchmark naive, filtered-adaptive, and fully exact orientation. Confirm the roughly $2\times$ average figure.
- 6 Implement the incircle predicate with a filter, and test on cocircular input.
- 7 Re-run Module 01's cyclic-consistency check against the filtered predicate and confirm it now passes everywhere.
- 8 **Implement symbolic perturbation** for orientation and verify it never returns zero.
- 9 Compile your predicate with `-ffast-math` and find an input where it now gives the wrong sign.
- 10 Compare your implementation against `predicates.c` on correctness and speed.

Self-check

Answer these in writing, from memory, without looking back at the notes. If you cannot, you have read this module but not yet learned it — that is useful information, not a failure.

- 1 Why is the sign of a geometric predicate a decidable problem?
- 2 What does a floating-point filter certify, and how does that differ from an epsilon?
- 3 What is an expansion, and why does exact arithmetic need no bignum library?
- 4 Describe adaptive evaluation and its average cost.
- 5 State the incircle predicate and say what its zero case means.
- 6 Explain symbolic perturbation and what question it actually answers.
- 7 Give three ways to handle degeneracy and when each is right.
- 8 Name three compiler behaviours that break an exact predicate.